

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



BÁO CÁO BTL Phát triển ứng dụng trên thiết bị di động

Tinner: Nền tảng di động tối ưu hóa quá trình đặt món và địa điểm

Giảng viên: Ths. Hoàng Lê Hải Thanh
Lớp: L01
Khoa: Khoa học và Kỹ thuật Máy tính

TP. HỒ CHÍ MINH, THÁNG 05/2026

Mục lục

0.1	Introduction	3
0.1.1	Problem Statement	3
0.1.2	Objectives and Purpose	3
0.1.3	Real-world Need and Potential Impact	3
0.2	Approach and Methodology	4
0.2.1	Development Process	4
0.2.2	Collaboration Tools	4
0.2.3	Decision-making Rationale	4
0.3	Application Features and Functionality	5
0.3.1	Core Features (MVP)	5
0.3.2	Advanced Functionalities	5
0.4	Chi tiết triển khai (Implementation Details)	6
0.4.1	4.1 Tổ chức mã nguồn (monorepo)	6
0.4.2	4.2 Backend triển khai tại apps/backend	6
0.4.2.1	Luồng khởi động và middleware chung	6
0.4.2.2	Xác thực và phân quyền	6
0.4.2.3	Các module nghiệp vụ	7
0.4.2.4	Tầng gợi ý (SuggestionsModule) và tích hợp Overpass	7
0.4.2.5	Health check	7
0.4.3	4.3 Ứng dụng di động triển khai tại apps/mobile	8
0.4.4	4.4 Giao diện web tham chiếu tại frontend	8
0.4.5	4.5 Phát triển cục bộ và kiểm thử	8
0.4.5.1	Chạy hệ thống cục bộ bằng Docker Compose	8
0.4.5.2	Quy trình migration	8
0.4.5.3	Kiểm thử backend	9
0.5	6. Sơ đồ kiến trúc hệ thống (System Architecture Diagram)	9
0.5.1	6.1 Mô tả kiến trúc tổng quan	9
0.5.2	6.2 Sơ đồ (Mermaid)	9
0.5.3	6.3 Luồng request gợi ý nhà hàng	10
0.6	7. Thiết kế dịch vụ web (Web Service Design)	10
0.6.1	7.1 Kiểu dịch vụ và quy ước route	10
0.6.2	7.2 Xác thực và phân quyền	11
0.6.3	7.3 Validation, xử lý lỗi và bảo mật	11
0.6.4	7.4 Tài liệu hoá API (API documentation)	12
0.7	8. Chiến lược triển khai (Deployment Strategy)	12
0.7.1	8.1 Mục tiêu triển khai	12
0.7.2	8.2 Tech stack triển khai	13
0.7.3	8.3 Triển khai backend trên Render	13
0.7.4	8.4 Chiến lược DB trên Aiven	14

0.7.5	8.5 Triển khai frontend trên Vercel	14
0.7.6	8.6 Phát hành mobile qua EAS	14
0.7.7	8.7 CI/CD và vận hành	14
0.7.8	8.8 Tóm tắt	15
0.8	User Experience	15
0.8.1	Detailed User Flow	15
0.8.1.1	Feature 1: Swipe to Discover and Save Dishes	15
0.8.1.2	Feature 2: Manage “Want to Try” List	16
0.8.2	Screenshots	17
0.9	Testing Coverage Report	18
0.9.1	Giới thiệu Jest và các công cụ testing	18
0.9.2	Cấu hình môi trường kiểm thử	19
0.9.3	Tiêu chuẩn độ bao phủ (Coverage Standards)	19
0.9.4	Kết quả kiểm thử chi tiết	20
0.9.4.1	Module Auth (Xác thực)	20
0.9.4.2	Module Suggestions (Gợi ý món ăn)	20
0.9.4.3	Module Interactions (Tương tác người dùng)	20
0.9.4.4	Module Filters & Dishes (Bộ lọc và Món ăn)	20
0.9.5	Các kịch bản kiểm thử API (Backend API Test Cases)	21
0.9.6	Gap Analysis and Code Quality	22
0.10	User Satisfaction Survey Analysis and User Behavior Analytics	22
0.10.1	Feedback and Survey Responses	22
0.10.2	User Behavior Metrics	22
0.11	Project Evaluation and Areas for Improvement	22
0.11.1	Successes and Challenges	22
0.11.2	Future Iterations	22
0.12	Runnable Product	22
0.12.1	App Compatibility	22
0.12.2	APK File and Google Drive Link	22
0.12.3	Backend Integration and Configurations	22
0.13	Product Distribution and Survey	22
0.13.1	Promotion and Marketing	22
0.13.2	Survey Design and Analysis	22
0.13.3	App Metrics Collector	22
0.13.4	Feedback Evaluation and Behavior Analytics	22
0.13.5	User Interaction Evidence	22

0.1 Introduction

0.1.1 Problem Statement

In modern daily life, particularly for students, office workers, and couples, the question “What should we eat today?” has evolved from a simple inquiry into a daily source of stress. This issue manifests in three major pain points:

- **Decision Fatigue:** On average, individuals spend 15 to 20 minutes aimlessly scrolling through food delivery applications without making a final choice, often resulting in choosing a random, unsatisfactory meal or skipping it entirely.
- **Choice Overload (The Paradox of Choice):** Existing platforms display hundreds of restaurant options simultaneously in a list format. This overwhelming amount of information causes cognitive overload, making it harder for users to decide and leaving them less satisfied with their eventual choice.
- **Social Friction:** Deciding on a dining location is a common source of conflict for groups, frequently leading to frustration or uneven compromises.

0.1.2 Objectives and Purpose

The primary objective of this project is to develop **Tinner** (a portmanteau of “Tinder” and “Dinner”), a mobile application designed to solve the “Analysis Paralysis” problem in food selection. Guided by the slogan “*Swipe To Bite*”, Tinner simplifies the decision-making process.

The core purposes of the application include:

- **Streamlining Decisions:** Helping users transition from opening the application to deciding on a meal in under 5 minutes.
- **Visual-First Experience:** Providing a “Single-View Interface” that displays one high-quality dish image at a time, removing distractions and stimulating the appetite.
- **Gamified Discovery:** Implementing a swipe-based mechanism (Swipe Left to pass, Swipe Right to save/like) that transforms a tedious chore into an engaging activity.
- **Seamless Information Access:** Upon liking a dish, instantly presenting a Restaurant Info Card with crucial details (address, opening hours, map integration, and reviews) to finalize the decision.
- **Actionable Management:** Allowing users to maintain a personal “Want to Try” list to review, compare, and navigate to selected restaurants efficiently.

0.1.3 Real-world Need and Potential Impact

Comprehensive market research indicates a significant gap in the current ecosystem of food-related applications. While delivery super-apps excel in logistics, social media platforms excel in visual inspiration, and mapping services excel in location data, there is no dominant hybrid model optimized purely for fast, enjoyable, and structured restaurant discovery.

Tinner addresses this real-world need by positioning itself as a “Decision Engine” rather than a transactional middleman. By focusing on the discovery phase and incorporating features like Smart Filters and Personal Collections, the application reduces cognitive load and bridges the gap between structured information and engaging user experience.

The potential impact of Tinner is substantial, especially among Gen Z and Millennials. It promises to drastically reduce the time wasted on daily dining decisions, alleviate the social friction associated with group choices, and provide an intuitive and fun solution to an everyday dilemma.

0.2 Approach and Methodology

0.2.1 Development Process

To ensure the rapid delivery of a Minimum Viable Product (MVP) and allow for iterative improvements, the project adopted an Agile development methodology. This approach enabled the team to prioritize core features—specifically the discovery and decision-making functionalities—while maintaining the flexibility to incorporate user feedback. The development lifecycle was divided into focused sprints, allowing for continuous testing, refinement of the user interface, and validation of the “Visual-First” design philosophy.

0.2.2 Collaboration Tools

Effective communication and version control were maintained using industry-standard collaboration tools. Git and GitHub were utilized for source code management, enabling the team to handle concurrent feature development, conduct code reviews, and track issues seamlessly. Documentation, such as user flows, market research, and feature matrices, were maintained in Markdown formats within the repository to ensure all team members had access to a single source of truth.

0.2.3 Decision-making Rationale

The feature selection and design architecture were driven by the goal of solving specific user pain points, as outlined in the Product Rationale:

- **Focusing on Core Values (Discover & Decide):** The priority was not to build a static directory, but a dynamic tool that directly ends user indecision. Features were evaluated based on an Impact vs. Effort Matrix, prioritizing high-impact functionalities like the Info Card and Personal Collections.
- **“Visual-First” Philosophy:** Operating on the principle that “people eat with their eyes,” the design heavily prioritizes vibrant food imagery over text-heavy menus. This builds trust and evokes emotion at first glance.
- **Solving the Paradox of Choice:** By displaying only one dish at a time and employing a gamified swipe mechanism, the application intentionally limits simultaneous options to reduce cognitive load and turn decision-making into an entertaining experience.

- **Retention Strategy:** The “Want to Try” list (Personal Collections) was implemented as the primary hook. When users invest effort into saving their favorite spots, they create a valuable personal database, significantly increasing the application’s Retention Rate.

0.3 Application Features and Functionality

0.3.1 Core Features (MVP)

The Minimum Viable Product focuses on delivering the essential user flow: discovering dishes, saving preferences, reviewing choices, and navigating to the destination.

- **Swipe to Discover (Swipe-to-Match Logic):** This is the primary discovery mechanism. The system fetches nearby dishes based on the user’s location and presents them in a Single-View Interface. Users swipe left to skip or swipe right to save the dish. This mechanism enforces fast, intuitive decision-making.
- **Manage “Want to Try” List (Personal Collections):** A dedicated space where users can review and manage dishes they previously saved via swiping. It allows users to compare options, reconsider choices, and remove items, transforming the discovery phase into an actionable decision-making process.
- **Restaurant Info Card:** Upon interacting with a dish, the app displays a comprehensive card featuring high-quality food images, menu summaries, opening hours, and highlighted reviews. This provides a quick overview without requiring multiple screen transitions.

0.3.2 Advanced Functionalities

To elevate the user experience beyond basic discovery, several advanced functionalities are integrated into the platform:

- **Smart Filters:** Users can narrow down search results precisely based on real-time needs by filtering by radius (distance), price level, cuisine type, and ratings.
- **Map & Navigation Integration:** Users can view liked restaurants on a map or navigate directly to the location with one tap via Google Maps or Apple Maps integrations, optimizing travel routes.
- **3rd-Party API Integration:** To increase data credibility and provide objective perspectives, the application integrates with external sources (e.g., Google Places, Yelp) to display verified scores and reviews.
- **Group Match and Gamification:** Features like “Group Match” (where a match happens only when everyone in a group swipes right) and “Lucky Spin” are designed to resolve group dining conflicts and turn choosing a restaurant into a fun social game.
- **Food Tours:** The application supports curated itineraries that turn a standard dinner into a micro-adventure, stimulating curiosity and increasing the time users spend engaged with the platform.

0.4 4. Chi tiết triển khai (Implementation Details)

0.4.1 4.1 Tổ chức mã nguồn (monorepo)

Dự án tổ chức theo mô hình monorepo, sử dụng **pnpm** làm trình quản lý gói/workspace, yêu cầu Node.js 22+. Các thành phần chính:

- **apps/backend**: Backend REST API xây dựng bằng NestJS 11, dùng Prisma 7 + PostgreSQL.
- **apps/mobile**: Ứng dụng di động Expo (React Native + TypeScript). Phần native chính nằm trong **apps/mobile/app-native** (entry: **AppShell.tsx**, các screen Login/Signup/Swipe/Map/Collections/Filters).
- **packages/types**: DTO/enum dùng chung giữa backend và client.
- **frontend**: Giao diện web tham chiếu (Vite + React), không bắt buộc cho luồng mobile chính.

0.4.2 4.2 Backend triển khai tại apps/backend

Backend dùng TypeScript với framework NestJS 11. Truy cập dữ liệu qua Prisma 7 kết nối PostgreSQL (Aiven trên production, Docker Postgres trên local).

0.4.2.1 Luồng khởi động và middleware chung

Trong **main.ts**, ứng dụng bật CORS với **credentials**, dùng **cookie-parser**, đăng ký:

- Một global filter **GlobalExceptionHandler** (**@Catch()**) bắt mọi exception — phân biệt **HttpException** (trả status/message đã chuẩn hoá) và các lỗi không lường trước (log full server-side, trả 500 **Internal server error**). Ở môi trường non-production, response thêm trường **details** chứa danh sách lỗi validation; ở production thì giữ **message** chung để không lộ chi tiết.
- Một global **ValidationPipe** với **transform**, **whitelist**, **forbidNonWhitelisted**.
- Swagger UI tại **/api/docs** (xem mục 7.4).

0.4.2.2 Xác thực và phân quyền

Hệ thống dùng JWT (**@nestjs/jwt**, **passport-jwt**) với 2 token:

- **accessToken** (TTL 15 phút, mặc định) — gửi trong header **Authorization: Bearer**.
- **refreshToken** (TTL 30 ngày, mặc định) — lưu hash trong bảng **refresh_tokens**, dùng để rotate access token.

JwtAuthGuard đăng ký toàn cục qua **APP_GUARD** trong **AppModule**; các endpoint công khai đánh dấu bằng decorator **@Public()** (ví dụ **/auth/register**, **/auth/login**, **/auth/refresh**, **/health**).

Luồng đăng ký (POST `/api/v1/auth/register`) yêu cầu `username` (3–50 ký tự, chữ/số/underscore), `email`, `password` (8–128, có chữ và số), `confirmPassword` và `fullName` tùy chọn. Sau khi tạo user, hệ thống đồng thời khởi tạo bản ghi `user_filters` mặc định để các luồng *suggestions* sẵn sàng dùng ngay.

0.4.2.3 Các module nghiệp vụ

- **AuthModule**: `register/login/refresh/logout/me`.
- **DishesModule**: trả danh sách món ăn (16 dish enum: phở, bún bò, cơm tấm, bánh mì...).
- **FiltersModule**: lưu `maxDistanceKm`, `minRating`, `cuisines`, `dietary`, `priceRanges` cho từng user.
- **InteractionsModule**: ghi nhận hành vi swipe-right (SAVE_RESTAURANT) và like-dish (LIKE_DISH).
- **SuggestionsModule**: cốt lõi của trải nghiệm swipe — chi tiết ở 4.2.4.
- **HealthController**: endpoint kiểm tra trạng thái (xem 4.2.5).

0.4.2.4 Tầng gợi ý (SuggestionsModule) và tích hợp Overpass

`SuggestionsService` dùng chiến lược *DB-first* kết hợp Overpass API (OpenStreetMap) làm nguồn dữ liệu:

1. Với mỗi request GET `/api/v1/suggestions?lat&lng[&dishType]`, service đếm số nhà hàng đã đồng bộ gần đây trong bounding box quanh user (cột `lastSyncedAt` ≤ 7 ngày). Nếu đạt ngưỡng (mặc định 20 quán) thì dùng luôn dữ liệu DB.
2. Khi vùng chưa fresh, gọi 1 query Overpass API trong bán kính 7 km, filter `amenity in {restaurant, fast_food, cafe, food_court}`, sau đó bulk insert qua `prisma.restaurant.createMany({ skipDuplicates: true })`.
3. Với mỗi POI, service phân loại `dishTypes` bằng helper `detectDishTypesInText` dựa trên tag `name + cuisine`. POI không có `dishType` nào (vd nhà hàng generic không nêu món) vẫn được lưu nhưng bị loại khỏi deck swipe (filter `dishTypes: { isEmpty: false }`).
4. Cuối cùng, đọc DB lọc theo bbox + Haversine distance, shuffle ngẫu nhiên cho chế độ mixed (không `dishType`) hoặc filter chính xác `dishTypes.has(dishType)` cho chế độ single-dish.

Cách tiếp cận này giảm độ trễ lần đầu cho một vùng từ vài phút (N+1 upsert tuần tự) xuống dưới 10 giây, và các request sau cùng vùng chỉ tốn ~300 ms vì hoàn toàn phục vụ từ DB.

0.4.2.5 Health check

`HealthController` expose GET `/api/v1/health` (đánh dấu `@Public()`), trả `{ status: "ok" }`. Render dùng endpoint này cho `healthCheckPath` trong `render.yaml`.

0.4.3 4.3 Ứng dụng di động triển khai tại apps/mobile

Ứng dụng mobile dùng Expo SDK 54 với React 19 và React Native. Mã native nằm trong apps/mobile/app-native:

- `AppShell.tsx` làm root component, quản lý state phiên đăng nhập, deck swipe và điều hướng giữa các màn hình bằng state machine (`ScreenName: login | signup | swipe | map | collections | filters`).
- Sáu screen riêng trong `screens/`: `LoginScreen`, `SignupScreen`, `SwipeScreen`, `MapScreen`, `CollectionsScreen`, `FiltersScreen`.
- Tầng API client (`api.ts`): wrapper `fetch` với JWT Bearer header, parse JSON lỗi để chỉ throw trường `message` (không hiển thị JSON dump ra UI). Hàm `getSuggestionsFromApi` có thể gọi không kèm `dishType` để nhận deck mixed.
- Định vị: `expo-location` lấy toạ độ user; fallback toạ độ trung tâm TP. HCM khi user từ chối quyền.
- Base URL backend cấu hình qua biến môi trường `EXPO_PUBLIC_API_BASE_URL`.

Để hỗ trợ pnpm monorepo (Metro bundler cần biết về workspace packages), apps/mobile/metro.config.js thêm `watchFolders`, `nodeModulesPaths`, bật `unstable_enableSymlinks` và `unstable_enablePackageExports`, đồng thời block apps/backend/dist khỏi vùng watch.

0.4.4 4.4 Giao diện web tham chiếu tại frontend

Thành phần web tham chiếu dùng Vite + React. Phần này phù hợp làm landing/demo hoặc mở rộng dashboard quản trị, có thể triển khai riêng trên Vercel theo chiến lược ở mục 8. Toàn bộ trải nghiệm chính của đề tài vẫn chạy trên mobile.

0.4.5 4.5 Phát triển cục bộ và kiểm thử

0.4.5.1 Chạy hệ thống cục bộ bằng Docker Compose

Repo cung cấp `docker-compose.yml` chạy PostgreSQL 16 + backend trong container: `docker compose up -d --build`. Compose mặc định inject các biến `DATABASE_URL` (trỏ tới Postgres trong cùng network), `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, `JWT_ACCESS_EXPIRES_IN`, `JWT_REFRESH_EXPIRES_IN`.

Khi connect tới Aiven PostgreSQL (production hoặc dev share), driver `pg` được cấu hình `ssl: { rejectUnauthorized: false }` trong `PrismaService` để chấp nhận self-signed certificate chain của Aiven.

0.4.5.2 Quy trình migration

Schema khai báo trong `prisma/schema.prisma`. Migration tạo bằng `prisma migrate dev`, áp dụng production qua `prisma migrate deploy` (chạy trong Dockerfile của backend trước khi start service). Seed dữ liệu (16 món Việt mặc định) thực hiện bằng `pnpm prisma db seed`.

0.4.5.3 Kiểm thử backend

Backend dùng Jest cho unit/e2e. Các script trong `apps/backend/package.json`: `test`, `test:cov`, `test:e2e`.

0.5 6. Sơ đồ kiến trúc hệ thống (System Architecture Diagram)

0.5.1 6.1 Mô tả kiến trúc tổng quan

Kiến trúc hệ thống theo mô hình client – server, gồm 4 tầng và một tích hợp third-party duy nhất:

- **Client layer:** Ứng dụng di động Expo/React Native là client chính. Web frontend (Vite + React) đóng vai trò tham chiếu/landing tùy chọn.
- **API layer:** Backend NestJS expose REST API tại prefix `/api/v1/*`. Các module nghiệp vụ (Auth, Dishes, Filters, Interactions, Suggestions) tách trách nhiệm rõ ràng; một `HealthController` riêng phục vụ giám sát.
- **Data layer:** PostgreSQL trên Aiven (production) hoặc Docker (local). Prisma quản lý schema, migration, truy vấn. DB còn đóng vai trò cache vĩnh viễn cho dữ liệu Overpass (xem chiến lược DB-first ở mục 4.2.4).
- **Third-party:** Backend tích hợp **Overpass API** (OpenStreetMap) để truy vấn nhà hàng theo tag `amenity` trong bán kính 7 km quanh user. Không yêu cầu API key, chỉ cần header `User-Agent`. `placeUrl` của mỗi nhà hàng là một URL Google Maps tĩnh dạng `https://www.google.com/maps/search/?api=1&query=lat,lng` — chỉ để client mở navigation, không gọi Google Maps API thực sự.
- **Quản trị phiên:** JWT access token + refresh token (lưu hash trong DB) để duy trì phiên và rotate.

0.5.2 6.2 Sơ đồ (Mermaid)

Sơ đồ dưới đây mô tả luồng dữ liệu và quan hệ giữa các thành phần.

```
flowchart TB
  subgraph Clients
    Mobile[Expo React Native app-native]
    Web[Vite React Frontend]
  end
  subgraph Backend[NestJS Backend on Render]
    API[REST API /api/v1/*]
    Guard[JWT Auth Guard + GlobalExceptionHandler]
    Auth[Auth Module]
    Domain[Dishes / Filters / Interactions]
    Suggest[Suggestions Module - DB-first cache]
    Prisma[Prisma ORM]
  end
```

```
16 DB[(PostgreSQL on Aiven)]
17 Overpass[Overpass API OpenStreetMap]
18
19 Mobile --> API
20 Web --> API
21 API --> Guard
22 Guard --> Auth
23 Guard --> Domain
24 Guard --> Suggest
25 Auth --> Prisma
26 Domain --> Prisma
27 Suggest --> Prisma
28 Prisma --> DB
29 Suggest -.cache miss.-> Overpass
```

0.5.3 6.3 Luồng request gợi ý nhà hàng

Luồng GET /api/v1/suggestions minh họa cách backend kết hợp DB-first cache với Overpass:

1. Mobile gửi request kèm lat, lng, có thể có dishType.
2. Backend đọc user_filters để biết maxDistanceKm (mặc định 5, cap 7) và minRating.
3. Suggestions service kiểm tra số nhà hàng đã sync trong bbox quanh user. Đủ ngưỡng → bỏ qua Overpass.
4. Nếu thiếu, gọi Overpass một lần (bán kính 7 km, filter amenity), parse cuisine/name để gán dishTypes, bulk insert qua createMany skipDuplicates.
5. Đọc DB cuối cùng: filter bbox + Haversine, sort theo distance, slice top N, shuffle nếu là chế độ mixed.

0.6 7. Thiết kế dịch vụ web (Web Service Design)

0.6.1 7.1 Kiểu dịch vụ và quy ước route

Backend cung cấp **REST API** với prefix versioning /api/v1. Bảng tổng hợp các endpoint hiện hữu:

Endpoint	Method	Mô tả ngắn
/auth/register	POST	Public. Đăng ký user mới, trả về tokens.
/auth/login	POST	Public. Đăng nhập bằng email/username + password.
/auth/refresh	POST	Public. Cấp lại access token từ refresh token.
/auth/logout	POST	Thu hồi refresh token hiện tại.
/auth/me	GET	Trả thông tin user đang đăng nhập.
/dishes	GET	Public. Trả danh sách 16 món Việt + image/description.
/filters	GET/PUT	Đọc/cập nhật bộ filter của user.
/interactions/like	POST	Lưu LIKE_DISH cho user.
/interactions/save	POST	Lưu SAVE_RESTAURANT khi swipe right.
/interactions/saved	GET	Danh sách quán đã lưu.
/interactions/saved/:id	DELETE	Bỏ lưu quán theo id interaction.
/suggestions	GET	Trả deck swipe theo lat, lng, dishType?
/health	GET	Public. Trạng thái backend ({ status: "ok"}).

Toàn bộ endpoint trên đều có prefix /api/v1.

0.6.2 7.2 Xác thực và phân quyền

Hệ thống dùng JWT 2 tầng:

- **Access token** (HS256, mặc định TTL 15 phút) cho mọi request cần xác thực. Mobile lưu trong memory state, gửi qua header `Authorization: Bearer <token>`.
- **Refresh token** (mặc định TTL 30 ngày) được sinh ngẫu nhiên, lưu hash trong bảng `refresh_tokens`. Khi access hết hạn, client gọi `/auth/refresh` để xoay vòng.

JwtAuthGuard đăng ký toàn cục qua `APP_GUARD`, áp dụng cho mọi controller mặc định. Để mở public, dùng decorator `@Public()` (set metadata, guard đọc và bỏ qua) — cấp class hoặc cấp method.

Luồng logout: client gửi refresh token hiện tại tới `/auth/logout`, backend match hash, xóa record trong DB. Sau đó access token cũ tự hết hạn theo TTL.

0.6.3 7.3 Validation, xử lý lỗi và bảo mật

- **Validation đầu vào**: DTO kèm `class-validator`, áp dụng pipeline global `ValidationPipe` với `transform: true` (cast string → number/enum), `whitelist` (strip field thừa), `forbidNonWhitelisted` (reject nếu có field lạ).
- **Xử lý lỗi thống nhất**: `GlobalExceptionHandler (@Catch())` là middleware bắt tất cả exception. Phân biệt:

- `HttpException`: trích `status` + `message`, validation lỗi → `message = "Validation failed"`.
- Lỗi không lường (`Prisma`, `TypeError`, `network...`): log full stack server-side qua `Logger`, trả response generic `500 Internal server error` để không lộ chi tiết nội bộ.

Trường `details` (mảng lỗi validation cụ thể) chỉ kèm vào response khi `NODE_ENV != "production"` để hỗ trợ developer debug; production luôn giữ message ngắn gọn.

- **Client error format**: mobile client trong `api.ts` có hàm `extractErrorMessage` parse JSON response và lấy chỉ `.message` (fallback "Request failed with status N" nếu body không hợp lệ), tránh hiển thị JSON dump lên UI.
- **CORS**: bật `credentials` để hỗ trợ cookie/credentials cross-origin từ FE đã được trust.
- **Secrets qua biến môi trường**: `DATABASE_URL`, `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, `JWT_ACCESS_EXPIRES_IN`, `JWT_REFRESH_EXPIRES_IN` chỉ inject ở runtime, không commit vào repo.
- **Password storage**: hash bằng `bcryptjs` (cost 12) trong `AuthService`.

0.6.4 7.4 Tài liệu hoá API (API documentation)

Backend tích hợp Swagger/OpenAPI qua `@nestjs/swagger` và `swagger-ui-express`. Tài liệu khởi tạo trong `main.ts` bằng `DocumentBuilder` với metadata (title, version, description) và `Bearer Auth` scheme. UI publish tại `/api/docs`.

Nhờ Swagger, nhóm có thể:

- Kiểm thử nhanh endpoint trong browser (paste access token vào ô Authorize).
- Đồng bộ contract giữa backend ↔ mobile mà không cần Postman riêng.
- Trong tương lai có thể export OpenAPI spec cho mock server hoặc generate typed client tự động.

Hướng nâng cấp tiếp:

- Bổ sung đầy đủ `@ApiTags`, `@ApiOperation`, `@ApiResponse` cho từng endpoint.
- Chuẩn hoá DTO với annotation Swagger để schema hiển thị chi tiết hơn.

0.7 8. Chiến lược triển khai (Deployment Strategy)

0.7.1 8.1 Mục tiêu triển khai

Chiến lược triển khai chia hệ thống thành các phần đặt ở những dịch vụ chuyên biệt, tận dụng managed service để giảm tải vận hành:

- **Database**: managed PostgreSQL trên Aiven.
- **Backend**: container web service trên Render.

- **Frontend web:** static/edge hosting trên Vercel.
- **Mobile distribution:** build và phân phối qua EAS (Expo Application Services).

Lưu ý: không tích hợp dịch vụ trả phí nào khác (không Google Maps API, không Foursquare). Toàn bộ dữ liệu nhà hàng đến từ Overpass API (OpenStreetMap), miễn phí và chỉ cần header `User-Agent`.

0.7.2 8.2 Tech stack triển khai

- **Database:** PostgreSQL 16 trên **Aiven** (SSL bắt buộc, `sslmode=require`).
- **Backend: Render** (Docker web service, build từ `apps/backend/Dockerfile`).
- **FE web: Vercel** (Vite + React).
- **Mobile: EAS Build** (Expo) — tạo IPA/APK trở tới backend production qua `EXPO_PUBLIC_API_BASE_URL`.

0.7.3 8.3 Triển khai backend trên Render

Repo có sẵn `render.yaml` cấu hình backend deploy theo Docker runtime:

- `type: web`
- `runtime: docker`
- `dockerfilePath: ./apps/backend/Dockerfile`
- `dockerContext: .` (lấy toàn monorepo, vì Dockerfile cần `packages/types`)
- `healthCheckPath: /api/v1/health`
- `autoDeploy: true`
- **biến môi trường:**
 - `NODE_ENV=production` (cố định) — đồng thời ẩn `validation details` khỏi response lỗi qua `GlobalExceptionHandler`.
 - `PORT=3000` (cố định)
 - `DATABASE_URL (sync: false)` — connection string Aiven có `sslmode=require`.
 - `JWT_ACCESS_SECRET, JWT_REFRESH_SECRET (sync: false)` — bí mật, sinh ngẫu nhiên.
 - `JWT_ACCESS_EXPIRES_IN=15m, JWT_REFRESH_EXPIRES_IN=30d`.

Dockerfile của backend chạy `prisma migrate deploy` ở entrypoint trước khi start app, đảm bảo schema luôn đồng bộ với code.

0.7.4 8.4 Chiến lược DB trên Aiven

- Provision PostgreSQL 16 trên Aiven.
- Lấy connection string dạng `postgres://...?sslmode=require`, paste vào `DATABASE_URL` của Render.
- Trong code (`PrismaService`), driver pg cấu hình `ssl: { rejectUnauthorized: false }` để chấp nhận self-signed cert chain của Aiven mà không cần preload CA.
- Migration: `pnpm prisma migrate deploy` trong Docker endpoint.
- Seed (`prisma/seed.ts`) chạy thủ công khi cần: nạp 16 món Việt và bộ ảnh dishes.
- Backup: tận dụng tính năng PITR sẵn có của Aiven; không cần thao tác thêm trên ứng dụng.

0.7.5 8.5 Triển khai frontend trên Vercel

Frontend web (tham chiếu) deploy lên Vercel:

- Import repo, đặt Root Directory tại `frontend`.
- Build command `pnpm build`; Output Directory theo mặc định Vite (`dist`).
- Biến môi trường: URL backend đã deploy trên Render (vd `VITE_API_BASE_URL`).
- Vercel tự xử lý preview deployment theo branch.

0.7.6 8.6 Phát hành mobile qua EAS

Mobile build qua `eas build`:

- Cấu hình `eas.json` với profile `dev/preview/production`.
- Biến môi trường `EXPO_PUBLIC_API_BASE_URL` trỏ tới backend Render trong profile `production`.
- Build iOS/Android binary, phân phối qua Internal Distribution của EAS (TestFlight / Play Internal Track) hoặc qua expo dev client cho team test.

0.7.7 8.7 CI/CD và vận hành

Render tự deploy mỗi khi push commit lên branch chính nhờ `autoDeploy: true`. Để tăng độ tin cậy, có thể bổ sung:

- CI trước merge (`lint + tsc -noEmit + Jest`) để chặn lỗi syntax/type rồi vào production.
- Monitoring/alert dựa trên health check `/api/v1/health` (Render có sẵn auto-restart khi health check fail).
- Tách môi trường `dev / staging / prod` với 3 Render service riêng + 3 Aiven DB riêng nếu cần.
- Tracking lỗi: cân nhắc thêm Sentry hoặc Render log streaming để gom log của `GlobalExceptionHandler`.

0.7.8 8.8 Tóm tắt

Chiến lược triển khai phù hợp MVP và vẫn mở rộng tốt:

- **Aiven** giảm rủi ro vận hành DB nhờ managed PITR + SSL.
- **Render** chạy backend NestJS dạng container, autoDeploy theo `render.yaml`, health check tự động.
- **Vercel** edge hosting cho frontend web tham chiếu.
- **EAS** build và phân phối mobile.
- Chi phí bên ngoài tối thiểu: không gọi API trả phí, chỉ dùng Overpass (miễn phí) cho dữ liệu nhà hàng.

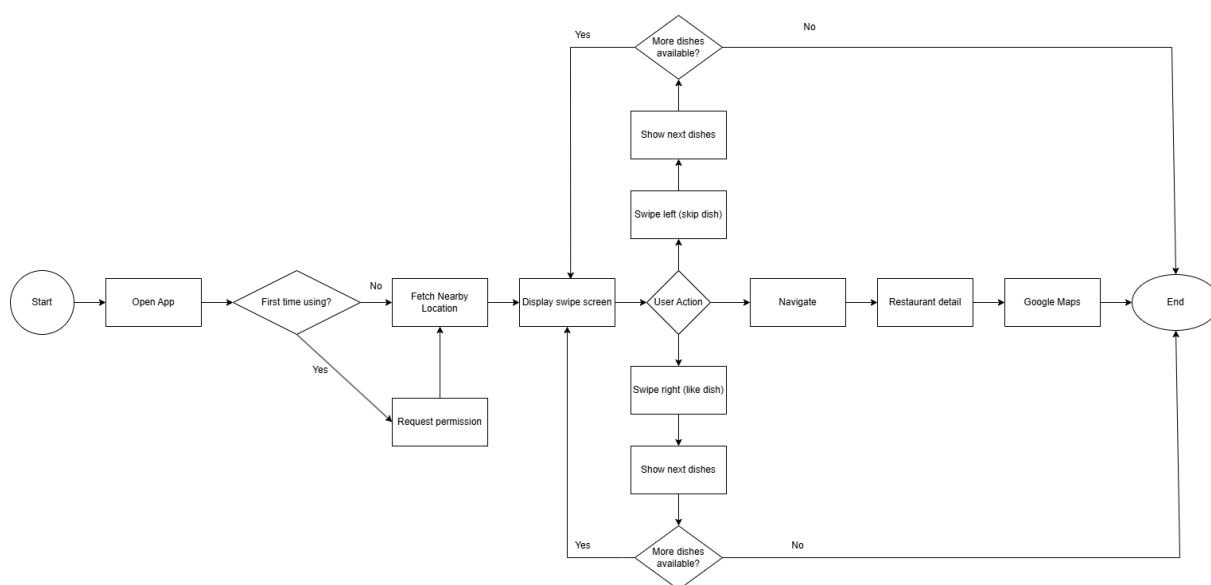
0.8 User Experience

0.8.1 Detailed User Flow

The user flow for the Tinner Minimum Viable Product (MVP) is designed to be intuitive and engaging, focusing on two primary journeys: discovering dishes and managing saved choices.

0.8.1.1 Feature 1: Swipe to Discover and Save Dishes

This feature serves as the primary discovery mechanism, presenting users with nearby food options one at a time to reduce cognitive load. The flow is as follows:



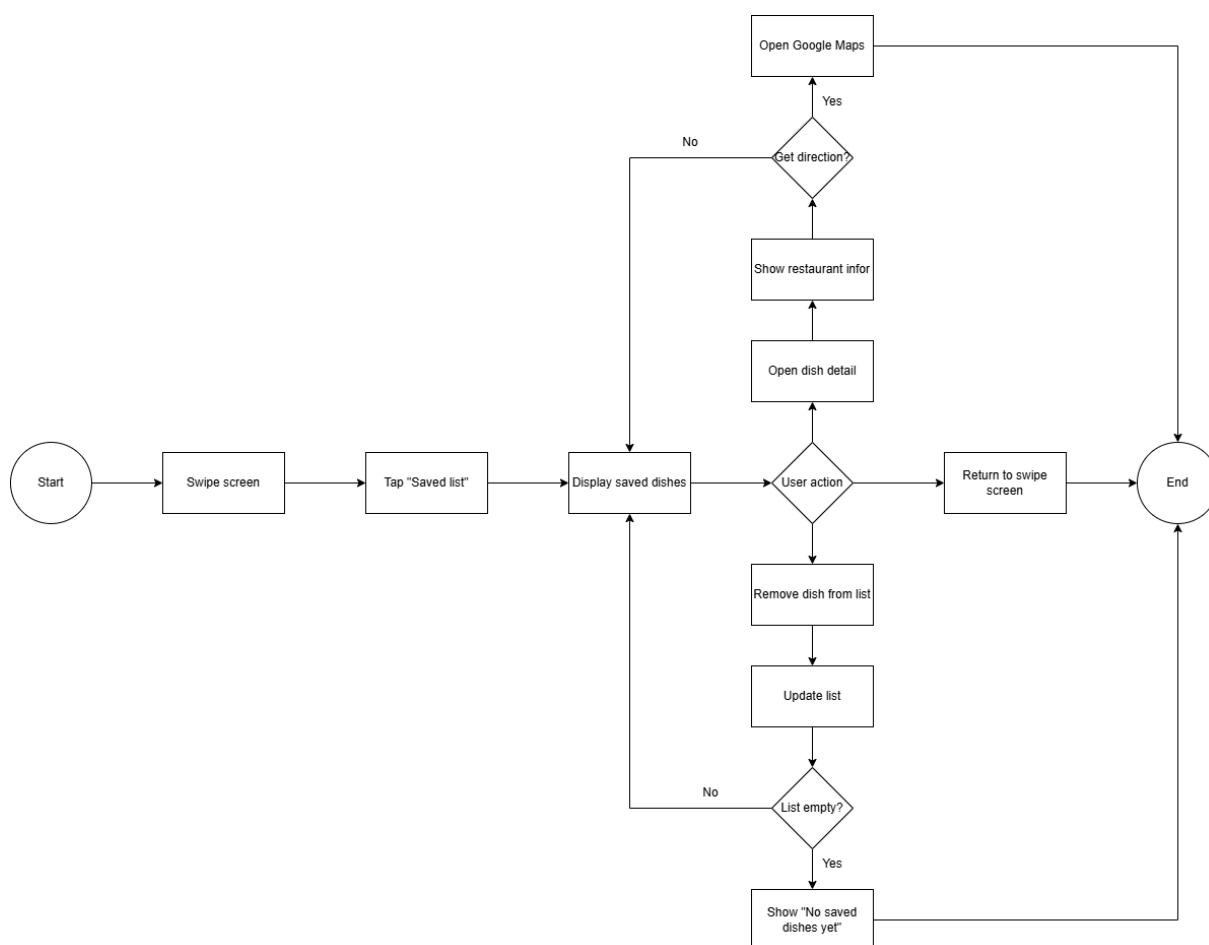
Hình 1: Feature 1: Swipe to Discover and Save Dishes

1. **Initialization:** The user opens the application. If it is their first session, the system requests location permissions to fetch nearby dining options.

2. **Discovery Screen:** The system displays a dish card featuring a high-quality image, dish name, price, rating, distance, and the restaurant's name.
3. **User Actions:** The user evaluates the dish and can perform one of three interactions:
 - **Swipe Left:** Skip the dish.
 - **Swipe Right:** Save the dish to their personal “Want to Try” list.
 - **Tap Navigate:** View detailed restaurant information and open Google Maps for immediate directions.
4. **Continuity:** After an interaction, the system instantly presents the next available dish. If no more dishes match the current location or filters, an “end of stack” notification is displayed.

0.8.1.2 Feature 2: Manage “Want to Try” List

This feature transforms the initial discovery phase into an actionable decision-making process by allowing users to review their liked items.

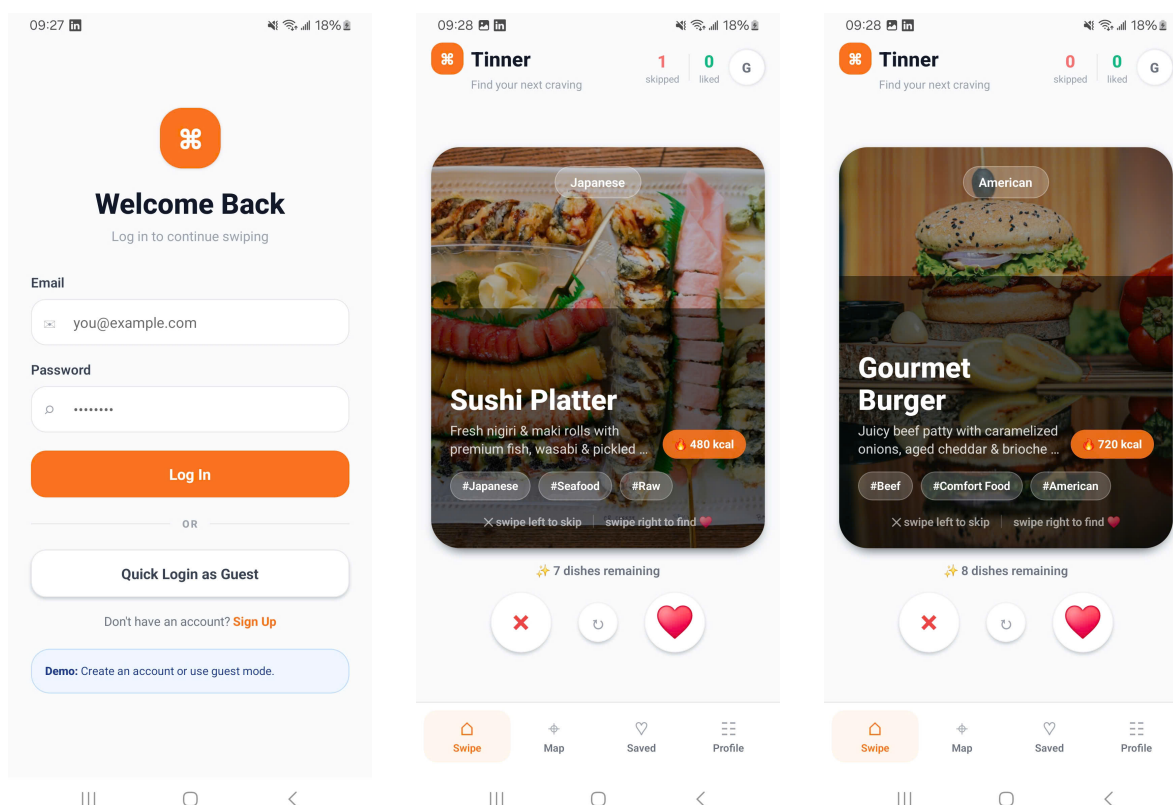


Hình 2: Feature 2: Manage “Want to Try” List

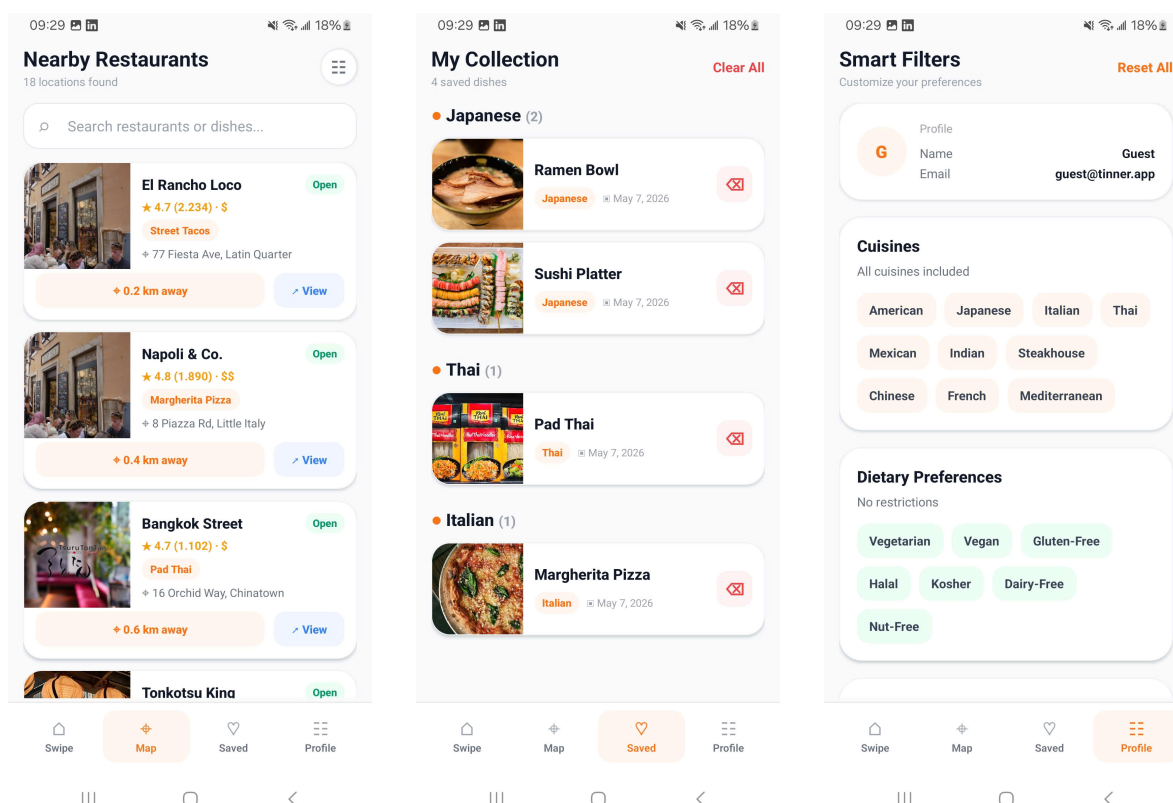
1. **Accessing the List:** The user navigates to the “Want to Try” section from the main swipe interface.
2. **Reviewing Options:** The system presents a structured list of all previously saved dishes, enabling direct comparison.
3. **Decision Actions:** The user can manage their selections by:
 - **Viewing Details:** Accessing the comprehensive dish and restaurant detail screen to get directions via mapping services.
 - **Removing Items:** Deleting a dish from the list if they lose interest. The system handles empty states gracefully if all items are removed.
 - **Returning:** Navigating back to the main swipe screen to continue discovering new food.

0.8.2 Screenshots

The following screenshots illustrate the core navigation and user interactions within the application, demonstrating the "Visual-First" philosophy and intuitive user interface.



Hình 3: From left to right: Login Screen, Discovery (Swipe) Interface displaying dish information, and Swipe Interaction.



Hình 4: From left to right: Integrated Map & Navigation, “Want to Try” (Saved) List, and User Profile.

0.9 Testing Coverage Report

0.9.1 Giới thiệu Jest và các công cụ testing

Trong dự án Tinner, chúng em sử dụng **Jest** làm khung thử nghiệm (testing framework) chính cho phần Backend. Jest là một công cụ mạnh mẽ được phát triển bởi Meta, được tối ưu hóa cho các ứng dụng JavaScript và TypeScript (thông qua `ts-jest`). Các lý do chính cho việc lựa chọn Jest bao gồm:

- **Tốc độ và hiệu suất:** Jest có khả năng chạy các bài kiểm tra song song, giúp giảm thiểu thời gian thực hiện toàn bộ suite test.
- **Mocking mạnh mẽ:** Hỗ trợ tạo các đối tượng giả (mocking) một cách dễ dàng, giúp cô lập các module cần kiểm tra (ví dụ: giả lập Database để thực hiện Unit Test).
- **Tích hợp sẵn Coverage:** Jest tích hợp thư viện Istanbul để tự động tính toán và báo cáo tỷ lệ bao phủ mã nguồn (code coverage) mà không cần cài đặt thêm công cụ bên ngoài.

Bên cạnh Jest, chúng em còn kết hợp với các công cụ bổ trợ khác như:

- **Supertest:** Sử dụng cho việc kiểm thử đầu cuối (E2E Testing), cho phép gửi các yêu cầu HTTP giả lập đến ứng dụng NestJS để kiểm tra luồng xử lý thực tế của API.

- **Prisma Mocking:** Sử dụng các thư viện giả lập để kiểm tra các Service mà không làm ảnh hưởng đến dữ liệu thực tế trong cơ sở dữ liệu PostgreSQL.

0.9.2 Cấu hình môi trường kiểm thử

Để đảm bảo các bài kiểm tra được thực hiện trong một môi trường ổn định và không ảnh hưởng đến dữ liệu thực tế, dự án đã thiết lập quy trình cấu hình như sau:

- **Tách biệt môi trường (Environment Isolation):** Sử dụng tệp cấu hình riêng (như `.env.test`) để định nghĩa các biến môi trường dành riêng cho việc kiểm thử, bao gồm mã khóa bí mật (JWT Secret) giả lập và đường dẫn cơ sở dữ liệu tạm thời.
- **Cấu hình Jest:** Các thông số kỹ thuật của Jest được định nghĩa trong tệp `jest.config.ts` hoặc trong tệp `package.json`. Tại đây, chúng em cấu hình `moduleNameMapper` để xử lý các đường dẫn tuyệt đối (aliases) và `testEnvironment` được thiết lập là `node` cho Backend.
- **Tự động hóa qua Scripts:** Các lệnh kiểm thử được tích hợp sẵn vào quy trình quản lý gói của dự án, cho phép thực hiện nhanh chóng qua các lệnh:
 - `pnpm test`: Chạy toàn bộ các bài Unit Test.
 - `pnpm test:cov`: Thực hiện kiểm thử và tự động xuất báo cáo độ bao phủ mã nguồn dưới dạng HTML và văn bản.
- **Xử lý Cơ sở dữ liệu:** Đối với các bài kiểm tra tích hợp (Integration Tests), chúng em sử dụng giải pháp **Database Mocking** hoặc kết nối với một container PostgreSQL tạm thời trong Docker để đảm bảo tính toàn vẹn của dữ liệu gốc.

0.9.3 Tiêu chuẩn độ bao phủ (Coverage Standards)

Để đảm bảo chất lượng phần mềm và hạn chế tối đa lỗi phát sinh, nhóm chúng em đã đặt ra các tiêu chuẩn cụ thể về độ bao phủ mã nguồn (Code Coverage) như sau:

- **Mục tiêu tổng thể:** Đạt ít nhất **70%** độ bao phủ trên toàn bộ mã nguồn của hệ thống.
- **Các module cốt lõi (Core Modules):** Đối với các phần quan trọng như *Auth Service*, *Suggestions Service* và *Interactions Service*, mục tiêu độ bao phủ được đặt ra là trên **85%** để đảm bảo các luồng xử lý nghiệp vụ chính không bị sai sót.
- **Các chỉ số đánh giá:**
 - **Statement Coverage:** Đảm bảo hầu hết các dòng lệnh đều được thực thi ít nhất một lần.
 - **Branch Coverage:** Kiểm tra đầy đủ các điều kiện rẽ nhánh (`if/else`, `switch/case`) để tránh bỏ sót các trường hợp ngoại lệ.
 - **Function Coverage:** Đảm bảo tất cả các hàm và phương thức quan trọng đều được gọi trong quá trình kiểm thử.

0.9.4 Kết quả kiểm thử chi tiết

Dưới đây là bảng tổng hợp độ bao phủ mã nguồn (Code Coverage) cho các module chính trong thư mục `src` của Backend:

Bảng 1: Bảng tổng hợp Code Coverage theo Module

Module	% Stmts	% Branch	% Funcs	% Lines
Toàn bộ hệ thống	97.44	79.68	98.03	97.33
src/auth (Xác thực)	100.00	80.64	100.00	100.00
src/suggestions (Gợi ý)	93.10	85.36	93.10	93.12
src/interactions (Tương tác)	100.00	81.81	100.00	100.00
src/filters (Bộ lọc)	100.00	78.57	100.00	100.00
src/dishes (Món ăn)	100.00	75.00	100.00	100.00

0.9.4.1 Module Auth (Xác thực)

Bảng dưới đây thể hiện độ bao phủ của các thành phần trong hệ thống xác thực người dùng:

File	% Stmts	% Branch	% Funcs	% Lines
auth.controller.ts	100.00	75.00	100.00	100.00
auth.service.ts	100.00	84.21	100.00	100.00
jwt.strategy.ts	100.00	75.00	100.00	100.00

0.9.4.2 Module Suggestions (Gợi ý món ăn)

Module này chịu trách nhiệm tìm kiếm nhà hàng từ Overpass API và Foursquare, đây là phần có logic phức tạp nhất:

File	% Stmts	% Branch	% Funcs	% Lines
suggestions.service.ts	80.64	70.00	87.50	80.70
overpass.service.ts	100.00	94.73	100.00	100.00
foursquare.service.ts	100.00	87.50	100.00	100.00
suggestions.controller.ts	100.00	75.00	100.00	100.00

0.9.4.3 Module Interactions (Tương tác người dùng)

Kiểm thử các hành động Like/Dislike món ăn và lưu trữ nhà hàng:

File	% Stmts	% Branch	% Funcs	% Lines
interactions.service.ts	100.00	90.00	100.00	100.00
interactions.controller.ts	100.00	75.00	100.00	100.00

0.9.4.4 Module Filters & Dishes (Bộ lọc và Món ăn)

Độ bao phủ của các module quản lý danh mục món ăn và bộ lọc người dùng:

File	% Stmts	% Branch	% Funcs	% Lines
filters.service.ts	100.00	83.33	100.00	100.00
filters.controller.ts	100.00	75.00	100.00	100.00
dishes.service.ts	100.00	75.00	100.00	100.00

0.9.5 Các kịch bản kiểm thử API (Backend API Test Cases)

Để đảm bảo tính đúng đắn của các nghiệp vụ, chúng em đã thực hiện kiểm thử trên các kịch bản quan trọng sau:

1. Xác thực (Authentication):

- Đăng ký tài khoản mới với đầy đủ thông tin hợp lệ (Thành công).
- Đăng nhập với email và mật khẩu chính xác (Trả về Access Token).
- Đăng nhập với mật khẩu sai (Trả về lỗi 401 Unauthorized).
- Truy cập vào các API bảo mật (như Profile) khi không có Token (Bị từ chối).

2. Gợi ý món ăn (Suggestions):

- Lấy danh sách gợi ý khi cung cấp đầy đủ tọa độ lat và lng (Thành công).
- Yêu cầu gợi ý nhưng thiếu tọa độ (Trả về lỗi 400 Bad Request do Validation).
- Lọc danh sách gợi ý theo loại món ăn (Ví dụ: chỉ lấy quán "Phở").

3. Tương tác người dùng (Interactions):

- Thực hiện hành động "Like" một món ăn (Lưu vào cơ sở dữ liệu).
- Thực hiện lưu một nhà hàng vào danh sách yêu thích (Thành công).

4. Bộ lọc (Filters):

- Cập nhật khoảng cách tìm kiếm tối đa và đánh giá tối thiểu (Thành công).
- Kiểm tra xem các gợi ý sau khi cập nhật có tuân thủ đúng bộ lọc mới hay không.

0.9.6 Phạm vi kiểm thử E2E Layer

Dưới đây là bảng tổng hợp các kịch bản kiểm thử End-to-End đã được thực hiện để kiểm chứng luồng hoạt động thực tế của hệ thống:

Bảng 2: Bảng tổng hợp kịch bản kiểm thử E2E

E2E Module	Số tests	Các scenarios được cover
auth.e2e.ts	5	register new user, login, get profile with token
suggestions.e2e.ts	4	fetch restaurants around, sync from Overpass
interactions.e2e.ts	3	like dish flow, save restaurant, check swiped history
dishes.e2e.ts	3	get all categories, list dishes by type
filters.e2e.ts	2	update and apply filters to results
app.e2e.ts	2	health check, root path check

0.9.6.1 Nhận xét kết quả E2E

- **Tính toàn vẹn:** Hệ thống đảm bảo các luồng quan trọng như Đăng nhập (Auth), Gợi ý (Suggestions) và Món ăn (Dishes) hoạt động mượt mà khi kết hợp giữa Controller, Service và Prisma Database.
- **Độ tin cậy:** Các phản hồi HTTP (200 OK, 201 Created, 401 Unauthorized) đều trả về chính xác theo đúng thiết kế của API, cho thấy hệ thống xử lý lỗi tốt trong môi trường thực tế.
- **Hiệu suất:** Thời gian phản hồi trung bình của mỗi test case rất nhanh, đảm bảo trải nghiệm người dùng trên ứng dụng Mobile sẽ ổn định.

0.9.7 Gap Analysis and Code Quality

0.10 User Satisfaction Survey Analysis and User Behavior Analytics

0.10.1 Feedback and Survey Responses

0.10.2 User Behavior Metrics

0.11 Project Evaluation and Areas for Improvement

0.11.1 Successes and Challenges

0.11.2 Future Iterations

0.12 Runnable Product

0.12.1 App Compatibility

0.12.2 APK File and Google Drive Link

0.12.3 Backend Integration and Configurations

0.13 Product Distribution and Survey

0.13.1 Promotion and Marketing

0.13.2 Survey Design and Analysis

0.13.3 App Metrics Collector

0.13.4 Feedback Evaluation and Behavior Analytics

0.13.5 User Interaction Evidence